Paso 3 — La plantilla: bloque fijo + hueco aleatorio, el recuento de condiciones y el registro de lo ya probado

Fecha: 2026-10-08 · **Para:** el dueño, que va a fijar los criterios él mismo · **Alcance:** sólo el paso 3 de `docs/AgentPDFs/WORKFLOW.md` («Plantilla — bloque fijo + hueco aleatorio, con registro de lo ya probado»): `sqx/templates/`, la skill `sqx-strategy-template`, el agente `templateArchitect`, las reglas duras 13 y 14 y las negativas de `sqx/projects/builder.py`. El vocabulario (paso 2), la paleta (paso 6) y el build quedan fuera salvo donde el recuento los necesita.

Revisado por dos miembros del Council (Feynman, Taleb) el 2026-10-08; correcciones aplicadas. Lo que viene de ellos y no he recalculado yo va marcado «calculado por el revisor», con sus salvedades.

Qué es: el documento de consulta del paso 3, con la forma del de los pasos 15-16 (`criterios-paso-16-spp-2026-10-07`). Es un paso de **autoría**, no de estadística: no juzga estrategias y no tiene reglas en `pipeline/autopilot/criteria.yaml`. Lo que tiene son **negativas** (el constructor se niega a crear el proyecto) y una **aritmética** (cuántas condiciones puede llevar cada estrategia). La única cuenta con sabor estadístico es la del tamaño del espacio de búsqueda, que se explica desde cero en la sección 2.

De dónde salen los números. Todo está recalculado por mí desde disco, sin tocar SQX:

- la librería `AlgoData/templates/library/` (61 plantillas), leída con el código del paso (`sqx.templates.holes.conditions`, `sqx.projects.builder.directions`, `sqx.projects.buildrules`);
- el `project.cfx` archivado de la corrida más reciente con plantilla, `Test_USDJPY_M30_doubleROCLong_10k_B5` (build del 2026-10-08, 01:24-02:57; `AlgoData/projects/retired/SQX_w2/`), leído con `sqx.projects.buildcensus`;
- las 41 estrategias de `doubleROCLong` en USDJPY M30 guardadas en el almacén (`AlgoData/store/USDJPY_M30_doubleROCLong/`, del 2026-10-07): son **supervivientes del paso 8**, no la población construida, y lo repito donde importa;
- el libro de búsqueda `AlgoData/ledger/USDJPY_M30_doubleROCLong.jsonl` y el registro de la campaña.

Las cuentas de combinaciones de las secciones 2 y 3.8 son **«calculado por mí»**: sus supuestos van al lado y los scripts están en la carpeta temporal `…/scratchpad/paso03/` (`census.py`, `built.py`, `space.py`, `tree.py`). «Ilustrativo» quiere decir cuenta de juguete para enseñar, no medida.

El commit que describo: `b2b4c8c`. **Con cambios sin confirmar en los ficheros del paso:** `sqx/templates/holes.py` (la función `conditions()` entera, que es la que cuenta), `sqx/templates/README.md`, `sqx/projects/buildrules.py` (`what_to_build` y `over_the_cap`); y **sin añadir a git**: `tests/test_condition_count.py` y toda la carpeta `pipeline/campaign/`. Es decir: **la aritmética de la regla 14 que describe este documento vive en el árbol de trabajo, no en el commit** (apartado 5.1).

Ninguna línea de este documento está congelada.

---

0. En una página

La pregunta del paso. De todo lo que el generador de SQX podría construir, ¿qué parte queda clavada por la idea (y es igual en todas las estrategias) y qué parte se deja al sorteo? Y, antes de gastar CPU: ¿esta plantilla ya se probó en este activo y esta temporalidad?

Lo cargado hoy. Ningún criterio estadístico. Tres negativas del constructor (`sqx/projects/builder.py:213-220`): dos direcciones, dos huecos libres en un lado, y fijas + hueco por encima de 2. Y una regla de escritura (`sqx/projects/buildrules.py:116-141`) que convierte lo que la plantilla fija en el rango «What to build» de la tarea Build.

Lo que sale en la librería real (61 plantillas, recuento mío con el código del paso):

forma en la entrada	plantillas	«What to build» que se escribe	condiciones de entrada que salen
1 fija literal + 1 hueco libre	37	0..1	1 o 2 (2 o 3 en las «dobles», ver 3.3)
1 hueco atado a un grupo + 1 hueco libre	18	1..1	exactamente 2
rechazadas por el constructor	6	—	—

De las 55 que pasan, 27 no tienen hueco de salida y 27 (`_withExit`) tienen un hueco libre de salida (0..2); una (`indicesIBS`) fija su salida.

Las ocho cosas que conviene saber antes de leer el resto:

1. **«What to build» no cuenta por estrategia cuando hay plantilla: cuenta por hueco.** SQX rellena *cada* `RandomCondition` con entre mín y máx condiciones. Por eso el rango escrito es `2 – fijas`, no 2. Es la regla 14 y está en `buildrules.py:139-140`.
2. **Una «doble» lleva dos o tres condiciones de entrada, no una o dos.** `doubleROCLong` fija dos veces el mismo bloque con periodos distintos; el código las cuenta como una (`holes.py:78-82` , decisión tuya del 2026-10-05), así que salen las dos ROC más 0 o 1 del hueco. Las 41 guardadas (supervivientes) tienen 3.
3. **Las 61 plantillas de la librería figuran como `draft` .** Ninguna ha pasado a `buildConfirmed` , aunque `doubleROCLong` se ha construido ocho veces. Ese estado lo cambian dos sitios, los dos a mano: la skill `/template-run` y la ventana (`ui/daemon/writes.py:10-30`); ni `bin/workflow.sh` , ni el autopiloto, ni la campaña lo tocan (5.3).
4. **`runs.csv` no es lo que evita repetir.** Guarda **una** fila por plantilla × activo × temporalidad y hoy tiene tres. Lo que de verdad impide repetir es el registro de la campaña (`pipeline/campaign/plan.py:67` y `:83`), y éste **no cuenta las celdas marcadas como prueba ni los proyectos `Test_`** (5.4).
5. **Hay tres N distintas y el libro sólo suma una.** (a) *Lógicas posibles*: la plantilla con su paleta admite **104** (el hueco vacío o una de 103 condiciones); es el tamaño de un espacio, no pruebas hechas. (b) *Estrategias generadas*: lo que el libro de búsqueda **suma** como N, **4.880.561** en ocho builds de `doubleROCLong` × USDJPY × M30; es lo que compitió en el tramo de build. (c) *N efectiva*: cuántas de ésas se comportan de forma distinta; el código la calcula sólo sobre supervivientes y nadie la ha medido sobre las generadas. Cuál de las tres debe corregir el resultado está **abierto** (2.6, 3.8, 6.3). Y la misma celda tiene además su `oos1` cribado cuatro veces con cuatro `criteria.yaml` distintos y 24 filas del libro sobre `oos2` (3.7, 5.12).

6. **En las 18 plantillas «de grupo» el hueco atado no es una idea fija: es un sorteo entre 2 y 13 bloques.** El código lo trata como «la condición fija del proyecto» (`buildrules.py:128-129`) porque así las emite `sqx.templates.build` (grupo de un solo bloque). Las de la librería actual no salieron de ahí: son tus ficheros, importados tal cual.
7. **La forma «atado + libre → 1..1» no se ha construido nunca con ese rango.** Ninguna de las 18 tiene un proyecto en `AlgoData/projects/registry.csv`. La ficha de conocimiento lo marca □ «sin probar en SQX».
8. **El constructor no mira dos cosas que la doctrina prohíbe o que rompen el build:** un bloque de calendario clavado en la plantilla (regla 15) y un bloque que lee un segundo gráfico. Lo primero sólo está anotado como aviso en el `manifest.json`; lo segundo abortó `filterMALong` en EURUSD M30 el 2026-10-07 y no dejó rastro en ningún registro de «ya probado» (5.5 y 5.6).

---

1. Qué es el paso y cómo decide

1.1 Dónde está

Tercer paso de la cadena: idea (1) → ¿existe el bloque? (2) → **plantilla (3)** → costes (4) → proyecto (5) → configuración del build (6). El paso 3 no arranca SQX ni gasta CPU: escribe un fichero `.sqx`, lo archiva en la librería y lo anota. Lo ejecuta la skill `sqx-strategy-template` (instalada en `~/.claude/skills/`, con las reglas del proyecto encima: `tools/sqx-lab/overlays/sqx-strategy-template.md`) o el agente `templateArchitect` (`.claude/agents/templateArchitect.md`, pasos 2-3 juntos, modelo Opus).

1.2 Las tres piezas de código

fichero	qué hace	lo llama
`sqx/templates/build.py`	mete **una** condición concreta en un esqueleto probado, dentro de un grupo de un solo bloque, y deja libre el segundo hueco	la skill, a mano
`sqx/templates/holes.py`	lee una plantilla y dice qué fija, qué ata a un grupo y qué deja libre; `conditions()` lo cuenta por lado	`buildrules`, `buildcensus`, la ventana
`sqx/templates/registry.py`	escribe una fila en `registry.csv` (plantillas) o en `runs.csv` (corridas)	la skill, la ventana, `/template-run`

1.3 Cómo «decide»

No hay veredicto Pasa / Limbo / No pasa. Hay dos momentos:

- **Al escribir la plantilla** (paso 3): la skill pregunta la lógica exacta (regla 11), aplica tus valores por defecto y verifica que el bloque aterrizó (`build.py:173-174`). No comprueba la regla 13 ni la 14.
- **Al crear el proyecto** (paso 5): `sqx.projects.builder` relee la plantilla y **se niega** si no cumple. Es aquí donde las reglas 13 y 14 se hacen cumplir. Una plantilla puede, por tanto, estar en la librería y ser inconstruible: hoy hay seis.

1.4 Tus valores por defecto (los aplica la skill y luego los dice)

`sqx/templates/README.md:18-27` y la cabecera de la skill:

lo que no dices	se asume
dirección	larga
condiciones extra	**una** aleatoria, libre (`#Group#` vacío)
lo que nombraste (la banda superior, p. ej.)	**fijo**, nunca parametrizado
periodos, desviaciones y cualquier número que no nombraste	**aleatorio**: el builder sortea uno por estrategia
`Shift`	1 (la última vela cerrada)

Lo único que nunca se asume es la lógica: «cierra por encima» y «cruza por encima» son estrategias distintas y se pregunta siempre.

---

2. Fundamentos: los conceptos que hacen falta

2.1 Condición, señal, regla

Una estrategia de SQX tiene **señales** (variables verdadero/falso: `LongEntrySignal`, `LongExitSignal`…) y **reglas** que actúan cuando una señal es verdadera («si `LongEntrySignal`, entra a mercado»). Una señal es un árbol de **condiciones** unidas por `AND`, `OR` o `Not`. Una condición es una frase que se evalúa en cada vela: «el ROC de 14 velas está por encima de 0».

Contar condiciones es contar las **hojas** de ese árbol. `AND(AND(a, b), c)` tiene tres.

2.2 Fija, atada, libre

Una plantilla es una estrategia a medio escribir. Cada hoja del árbol es de uno de tres tipos:

tipo	en el XML	qué pone el builder	la paleta la alcanza
fija (`fixed`)	el bloque escrito tal cual, p. ej. `ROCAboveLevel`	nada: sólo sortea los parámetros marcados `generate="random"`	no
atada (`bound`)	`RandomCondition` con `#Group#` = identificador de un grupo	un bloque **de ese grupo**	no
libre (`free`)	`RandomCondition` con `#Group#` vacío	un bloque de **todo lo encendido** en la tarea Build	**sí**: es lo único que gobierna

Un **grupo** es una lista de bloques con nombre. Un grupo de un solo bloque es el truco con que `sqx.templates.build` consigue «condición fija con periodos al azar»: SQX no deja marcar como aleatorio el parámetro de un bloque fijo escrito por nosotros («Identification not found», `build.py:59-62`), pero sí sortea los de cualquier bloque que salga de un grupo.

2.3 Simple Strategy frente a plantilla

	Simple Strategy	Plantilla
qué es	**sin fichero**: SQX arma la estrategia entera desde cero	un `.sqx` con parte clavada y parte abierta
cuándo	no hay idea fija; se quiere ver qué encuentra una familia de bloques	hay una idea que **todas** las estrategias deben llevar
cómo se pide	`builder … --simple long|short`	`builder … --template <librería>/template.sqx`
«What to build»	cuenta **por estrategia**: entrada 1..2, salida 1..2	cuenta **por hueco**: ver 2.4
condición de salida	**obligatoria** (mínimo 1)	optativa (mínimo 0)
dirección	un interruptor de la tarea (`MarketSides`)	la que tenga la plantilla
se registra como	`simple:long` en `projects/registry.csv` ; no hay fila en la librería	su carpeta en la librería

De aquí sale la frase de la regla 14: *una plantilla hecha sólo de huecos aleatorios es una Simple Strategy; se usa ésa y no se escribe plantilla*.

2.4 La trampa de «What to build»

En la pantalla del builder hay cuatro números: mínimo y máximo de condiciones de entrada, y lo mismo de salida. En una Simple Strategy significan lo que parecen. **Con plantilla, SQX aplica ese rango a cada hueco por separado y lo suma a lo que la plantilla ya fija.**

Ejemplo de juguete. Plantilla con una condición fija y un hueco, y el rango en 1..2 (lo que se escribía antes del 2026-10-02):

- el hueco recibe 1 o 2 condiciones → la estrategia lleva 1 + 1 = **2**, o 1 + 2 = **3**.

Para que el total no pase de 2, el rango del hueco tiene que ser `0..(2 − 1)` = 0..1. Con dos huecos libres y rango 1..2 salen de 2 a 4 condiciones. Con 0..1 por hueco saldrían de 0 a 2 (cabe una entrada vacía) y con 1..1 exactamente 2: la suma no pasa siempre de 2. El constructor rechaza los dos huecos **en cualquier caso** porque así lo dice la regla 14 («nunca dos huecos en un lado»; una plantilla sólo de huecos es una Simple Strategy): es una decisión tuya, no una consecuencia aritmética.

Evidencia de que SQX se comporta así (ficha `knowhow/authoring/condition-count-simple-vs-template.md`): con dos huecos libres y 1..2, el 69 % de una población USDJPY H1 traía tres o más condiciones de entrada; con un hueco atado + uno libre y 1..2, 1.962 de 5.026 traían un `AND` anidado como segunda condición.

2.5 Contar combinaciones, desde cero

Esto es lo que luego pesa en la corrección por pruebas múltiples, así que va despacio.

La regla del producto. Si una decisión tiene *a* opciones y otra, independiente, tiene *b*, juntas tienen *a* × *b*. Tres camisas y dos pantalones: seis conjuntos.

Elegir uno de C. Un hueco que recibe exactamente una condición de una paleta de C bloques tiene **C** opciones.

Elegir cero o uno. Si además puede quedar vacío: **1 + C**.

Elegir dos de C sin que importe el orden. `AND(a, b)` es lo mismo que `AND(b, a)`. Parejas distintas de C bloques: `C × (C − 1) / 2`. Con C = 10: 10 × 9 / 2 = **45**. A mano con C = 4 (a, b, c, d): ab, ac, ad, bc, bd, cd → 6 = 4 × 3 / 2.

Elegir uno o dos. Se suman: `C + C(C − 1)/2`. Con C = 10: 10 + 45 = **55**.

Dos huecos, cada uno con uno o dos. El producto directo es 55 × 55 = 3.025, pero **cuenta órdenes repetidos**: los dos huecos van unidos por `AND` y son intercambiables. Sin distinguir qué hueco es cuál: 55 × 56 / 2 = **1.540**. Y si además `{a} ∧ {b}` en dos huecos es lo mismo que `{a, b}` en uno, lo que queda son los conjuntos de 2, 3 o 4 bloques distintos: 45 + 120 + 210 = **375**.

Con esto, para una paleta de 10 bloques:

forma	cuenta	lógicas distintas
2 fijas, sin hueco	1	1
1 fija + hueco 0..1	1 + 10	11
hueco atado a un grupo de 4 + hueco libre 1..1	4 × 10	40
un hueco 1..2 (o Simple Strategy, sólo la entrada)	10 + 45	55
dos huecos 1..2 (rechazada)	55 × 56 / 2 (huecos sin orden); como conjuntos de 2 a 4 bloques	1.540; 375

Supuestos de toda cuenta de este tipo en el documento (mías): cuento *lógicas* (qué bloques, no con qué números); no cuento dos veces la misma pareja en otro orden; no cuento el mismo bloque repetido dentro de un hueco (no he verificado si SQX lo permite); y trato cada bloque de la paleta como una opción, aunque dos bloques puedan decir casi lo mismo. **No son cotas en sentido estricto:** un supuesto las desinfla (el bloque repetido que no cuento) y otro las infla (bloques casi sinónimos contados como distintos), y no he medido cuál pesa más. Son un orden de magnitud del número de *lógicas*; no dicen cuántos *comportamientos* distintos hay (38 estrategias de una misma lógica se parecen mucho entre sí, ver 3.3), y el número de estrategias con sus parámetros es muchísimo mayor.

2.6 Por qué importa: el mejor de muchos

Si pruebas N cosas que no valen nada y te quedas con la mejor, la mejor parece buena sólo por ser la mejor de N. Cuanto mayor N, mejor parece. Dos números ilustrativos, los dos para N resultados de puro ruido **independientes y de distribución normal**, medidos en desviaciones típicas *de los resultados entre sí* (no de una estrategia): la **cota** `√(2 × ln N)`, que el mejor casi nunca supera, y el **valor esperado del máximo**, que es lo que el azar entrega de media al mejor y queda algo por debajo. El proyecto usa lo segundo, en la forma de Bailey y López de Prado (`core/surface/plateau.py:55-71`, que lo dice: «sits below the sqrt(2 ln N) bound»).

N	E[máx] (lo que el azar da de media al mejor)	cota √(2 ln N)	qué N es
10	1,54	2,15	
104	2,52	3,05	las lógicas de `doubleROCLong` con su paleta
5.356	3,70	4,14	una entrada 1..2 con esa misma paleta de 103
165.600	4,49	4,90	una entrada 1..2 con las 575 condiciones
4.880.561	5,17	5,55	las estrategias generadas de verdad en la celda

E[máx] está calculado por mí por integración numérica (`space.py` no lo trae; la cuenta es `∫ x · N · φ(x) · Φ(x)^(N−1) dx`). Los dos revisores dan, con dos aproximaciones distintas, 1,55 / 2,51 / 3,68 / 4,48 / 5,15 y 1,57 / 2,54 / 3,71 / 4,50 / 5,17: coinciden a dos centésimas.

Tres lecturas. Una: el listón crece despacio (con el logaritmo), así que multiplicar el espacio por 50 no lo multiplica por 50, lo sube un escalón. Dos: con millones de intentos, un resultado a 5 desviaciones es justo lo que el azar **entrega** al mejor; deja de ser prueba de nada. De N = 104 a N = 4,9 millones el valor esperado pasa de 2,5 a 5,2 (cota: de 3,0 a 5,5). Tres: **esta tabla no es un umbral que se pueda fijar**, porque sus dos supuestos fallan aquí y en sentidos opuestos. Las estrategias vecinas no son independientes, y eso lo baja: con una correlación común ρ el máximo escala por √(1 − ρ). Y un Sharpe hecho de muchas ganancias pequeñas y pocas pérdidas grandes no es normal, y eso lo sube.

Calculado por el revisor (Taleb), no por mí ni por el estudio: en las 41 guardadas, PnL semanal sobre 446 semanas, las 38 de la misma lógica tienen correlación mediana **0,73** (mínimo 0,50) y equivalen a unas **2 independientes** (1,8 de 38 por razón de participación de autovalores); con ρ = 0,7 el 5,17 de arriba quedaría en 2,8. Salvedades suyas: son supervivientes del mismo cribado y todas largas en el mismo activo, así que parte del parecido lo pone el cribado; no mide la correlación entre las 4,9 millones generadas. En el otro sentido, su simulación de 200.000 estrategias de ruido con 300 operaciones da un máximo de 4,57 con operaciones normales, 4,04 con colas gruesas simétricas (t de 3 grados) y **6,58** con operaciones asimétricas; es una distribución de juguete (lognormal), no las operaciones reales.

Qué N es el correcto —lógicas posibles, estrategias generadas o N efectiva— queda abierto (3.8, 5.9), y no puedo acotarlo: ni 104 ni 4,9 millones son los extremos de un intervalo demostrado.

---

3. Lo que hace el paso, pieza a pieza

3.1 Emitir la plantilla: `sqx.templates.build`

```
python3 -m sqx.templates.build <nombre> <blocks.xml|config.xml> <clave> <salida.sqx> [--param '#Type#=1']
```

1. **Toma un esqueleto probado** (`build.py:14` , `:162`): por defecto `market_long` , un `.sqx` del kit con dos huecos `RandomCondition` unidos por `AND` .
2. **Convierte tu bloque en un elemento de señal** (`block_xml` , `:69-110`): cada parámetro numérico que no nombraste queda `generate="random"` ; lo que pasas con `--param` queda fijo. No se sortean el gráfico, el `Shift` ni los desplegables (tipo de media, precio): `optimizable` , `:34-47` .

3. **Lo mete en un grupo de un solo bloque** llamado `<nombre>Signal` (`group_xml`, `:50-66`). El identificador sale del nombre, así que reconstruir no lo cambia.
4. **Ata el primer hueco a ese grupo y vacía el segundo** (`bind_into_signal`, `:113-131`).
5. **Comprueba** que el XML se lee, que la clave del bloque está y que los huecos siguen siendo dos (`:172-174`); si no, falla.
6. Escribe `template.sqx` y `deps/groups.xml` (`:175-178`).

Resultado, en los términos del recuento: **1 atada + 1 libre** en la entrada, nada en la salida. Dirección: la del esqueleto.

Lo que este comando no hace: no comprueba la regla 13 ni la 14, no escribe `brief.md` ni `manifest.json`, no anota en `registry.csv` (lo hace la skill con `registry.py`) y no instala el grupo en los workers (paso aparte con `sqx.blocks.install`).

3.2 Leer la plantilla: `holes.conditions`

`sqx/templates/holes.py:85-110`. Para cada señal del fichero recorre su árbol (`_count`, `:69-82`):

- `AND`, `OR`, `Not` → se atraviesan;
- un `randomBlock` con `#Group#` relleno → **atada**; con `#Group#` vacío → **libre** (`:74-76`);
- cualquier otra cosa → **fija**, salvo que su «huella» (la tupla de claves de sus bloques) ya haya aparecido en esa señal: entonces no suma (`:78-82`).

El lado sale del nombre de la variable: si contiene `Exit` es salida; si no, entrada (`:106`). Si hay dos señales del mismo lado (larga y corta), cuenta la más llena (`:108`).

3.3 Una plantilla real abierta pieza a pieza: `doubleROCLong`

`AlgoData/templates/library/doubleROCLong/template.sqx` (5.602 bytes; tu fichero `DoubleROCLong_Template.sqx`, importado sin tocar el 2026-10-05). Dentro, un único XML de 197 líneas. Lo que dice, reducido a su árbol (script `tree.py`):

```
señal LongEntrySignal
  AND
    AND
      ROCAboveLevel   Periodo = ALEATORIO   Nivel = 0 (fijo)   Shift = 1 (fijo)
      ROCAboveLevel   Periodo = ALEATORIO   Nivel = 0 (fijo)   Shift = 1 (fijo)
    RandomCondition   #Group# vacío  → hueco LIBRE
regla «Long entry»
  si LongEntrySignal → EnterAtMarket, con ExitAfterBars ALEATORIO
(no hay señal de salida, ni regla «Short entry» que abra)
```

En castellano: «compra a mercado cuando el ROC de un periodo y el ROC de otro periodo están los dos por encima de cero **y** se cumple una condición más que elige el builder; cierra a las N velas».

Qué devuelve el código (recalculado): `conditions()` → entrada `{fixed: 1, bound: 0, free: 1}`, salida `{0, 0, 0}`. `directions()` → `['Long']`. `over_the_cap()` → vacío: se acepta.

Por qué `fixed` es 1 y no 2. Las dos hojas `ROCAboveLevel` tienen la misma huella; la segunda no suma. Es tu decisión del 2026-10-05: el mismo bloque dos veces con dos periodos es **una idea**. Consecuencia aritmética: el hueco recibe `2 − 1` = hasta 1 condición, y la estrategia sale con 2 ROC + 0 o 1 = **2 o 3 condiciones de entrada**.

Qué escribió el constructor en la tarea Build (del `project.cfx` archivado de `…_10k_B5`, tarea `Build-Task3.xml`):

```
minConditions="0" maxConditions="1" minExitConditions="0" maxExitConditions="2"
minPeriod="2" maxPeriod="200" minShift="1" maxShift="1"
```

y, del censo: sólo `EnterAtMarket`; salidas `ExitAfterBars` al 100 % (4 a 48 velas) y `_ExitRule_` al 50 %; **103 condiciones encendidas de 575**, 0 valores de 161, 0 niveles stop/limit de 85, 0 de reloj de 23 (paleta `doubleROCLong_USDJPY_M30`).

Qué salió. En las 41 estrategias guardadas de esta celda (supervivientes del paso 8; script `built.py`):

	resultado
condiciones de entrada	**3 en las 41** (2 ROC + 1 del hueco)
las dos `ROCAboveLevel` presentes	41 de 41
nivel del ROC	0 en las 82
periodos del ROC	de 10 a 120, siempre pares; ninguna pareja con los dos iguales; 22 parejas distintas
condiciones de salida	**0 en las 41**
`ExitAfterBars`	de 20 a 48
qué rellenó el hueco	`CBlock_WassersteinShiftHigh` en 38, `ADXCrossUp` en 2, `ATRPercentCrossDown` en 1

Tres cosas a retener. La plantilla se respeta: el bloque fijo está en todas. El hueco **nunca quedó vacío** en las guardadas, aunque el rango 0..1 lo permite — pero éstas son las que pasaron la puerta, así que no dice con qué frecuencia el builder lo deja vacío. Y 38 de 41 comparten **la misma lógica completa**: son una sola idea con 38 juegos de números (calculado por el revisor: correlación semanal mediana de 0,73 entre ellas; al ser supervivientes del mismo cribado y todas largas, parte de ese parecido lo pone el cribado, no la lógica).

3.4 La dirección: `builder.directions` (regla 13)

`sqx/projects/builder.py:30` y `:34-42`. Busca en el XML las reglas llamadas `Long entry` y `Short entry` y se queda con las que contienen una acción cuya clave empieza por `Enter`. Devuelve la lista de lados que abren.

La negativa (`:213-218`): si el proyecto va a construir (lleva tarea Build o es `--workflow`) y la lista no tiene exactamente un elemento, se para con «…abre operaciones en [Long, Short]: una plantilla, y un build, van en UNA sola dirección». La condición «si va a construir» existe porque el paso 26 pasa por el mismo constructor una estrategia ya generada, que trae acción de entrada en las dos reglas.

En la librería: 38 sólo largas, 18 sólo cortas y **5 con las dos** (`gbpRandomEntry`, `gbpRandomEntryExit`, `gbpRandomExit`, `spRSI2`, `xauWeekend`), que el constructor rechaza.

3.5 El recuento: `buildrules.what_to_build` y `over_the_cap` (regla 14)

El tope está en `assets/_build.yaml:23-28`: `max_entry_conditions: 2`, `max_exit_conditions: 2`, `simple: {entry: [1, 2], exit: [1, 2]}`.

La negativa (`over_the_cap`, `buildrules.py:144-166`), por cada lado:

- más de un hueco libre → rechazada, siempre y sea cual sea el rango que se fuera a escribir. El mensaje del código dice «con dos huecos salen hasta 4 condiciones; una plantilla de solo huecos es una Simple Strategy»; lo primero sólo es cierto con el rango 1..2 (con 1..1 saldrían exactamente 2): el rechazo es la política de la regla 14, no aritmética (2.4);
- si no, `fijas + atadas + libres > 2` → rechazada («con 2 fijas no cabe hueco»).

El rango que se escribe (`what_to_build`, `:116-141`), por cada lado:

- sin plantilla: lo de `complexity.simple`, tal cual (`:133-134`);
- con plantilla: `máx = 2 − fijas − atadas`, nunca menos de 0 (`:139`); y
- si el lado tiene algún hueco **atado**: se escribe **1..1** (`:140`);
- si no: `mín = 0` si hay alguna fija; si no hay ninguna, el suelo del lado (entrada 1; salida 0, o 1 si `exits.condition_required` es verdadero — hoy es falso).

Por qué 1..1 con un hueco atado. La tarea Build guarda **un solo rango por lado** (un `minConditions`/`maxConditions` en `<Chart name="Main chart">`, lo he visto en el `project.cfx`), así que vale para todos los huecos de ese lado. Que SQX lo aplique a cada hueco está medido con el rango 1..2 (2.4); que con 1..1 haga lo que se espera sobre un hueco atado está en la ficha como □, **sin probar en SQX** (5.9). El hueco atado es «la idea»; si el rango fuese 0..1, SQX podría dejar la idea fuera. Con 1..1 cada hueco recibe exactamente una: la idea y una aleatoria.

La tabla de casos trabajados. Cada fila está comprobada contra el código: las marcadas ✔ por `tests/test_condition_count.py` (17 comprobaciones, las he ejecutado hoy: «all ok»); las marcadas ● sobre plantillas reales de la librería.

#	lo que tiene el lado	recuento (fijas, atadas, libres)	¿se acepta?	rango escrito	condiciones que salen	dónde
1	Simple Strategy, entrada	—	sí	1..2	1 o 2	✔
2	Simple Strategy, salida	—	sí	1..2	1 o 2	✔
3	1 fija + 1 hueco libre	1, 0, 1	sí	**0..1**	1 o 2	✔ • `aroonLong`, `cciPercLong`, `squeezeBBLong`…
4	la misma fija dos veces + 1 hueco libre	1, 0, 1	sí	0..1	**2 o 3**	✔ • `doubleROCLong`, `doubleVortexLong`
5	1 atada + 1 libre	0, 1, 1	sí	**1..1**	exactamente 2	✔ • `trendRegimeFilters`, `bookTriggers`…
6	sólo 1 hueco libre (entrada)	0, 0, 1	sí	1..2	1 o 2	✔
7	sólo 1 hueco libre (salida)	0, 0, 1	sí	0..2	0, 1 o 2	• las 27 `_withExit`
8	2 fijas distintas, sin hueco	2, 0, 0	sí	0..0	exactamente 2	✔
9	2 fijas distintas + 1 hueco	2, 0, 1	**no**	—	(serían 2 o 3)	✔ • `gbpjpyEndMonth`
10	2 huecos libres	0, 0, 2	**no**	—	(serían 2 a 4)	✔
11	3 huecos atados	0, 3, 0	**no**	—	—	✔ esqueleto `role_market`
12	1 fija + 1 hueco libre, en la salida	1, 0, 1	sí	0..1	1 o 2	✔
13	1 fija en la salida, sin hueco	1, 0, 0	sí	0..1	exactamente 1	• `indicesIBS`
14	nada en la salida	0, 0, 0	sí	0..2	0 (ver nota)	• las 27 sin `_withExit`

Notas a la tabla:

- **Fila 4.** Es la excepción escrita en la regla 14. Si un día fijas *tres* veces el mismo bloque, el código seguirá contando una.
- **Filas 13 y 14.** El rango se escribe aunque no haya hueco que rellenar. En las 41 guardadas de `doubleROCLong` (fila 14, con `_ExitRule_` al 50 %) no hay ni una condición de salida: es coherente con que sin hueco SQX no añade nada, pero es una observación sobre supervivientes, no una prueba (5.9).
- **Fila 6 frente a la regla 14.** Una plantilla con un solo hueco libre y nada fijo se acepta. La regla dice que «una plantilla de nada más que huecos aleatorios es una Simple Strategy: se usa ésa». El código sólo rechaza **dos** huecos; con uno, deja pasar (5.7).

3.6 Archivar y anotar

La carpeta (`AlgoData/templates/library/<nombre>/`): `template.sqx` , `brief.md` (en castellano), `manifest.json` , y `deps/` con los bloques y grupos que referencia, para que se instale en cualquier SQX. Un `.sqx` sólo lleva referencias; copiado a una instalación sin sus bloques no da error, desaparece del builder en silencio.

El nombre: camelCase, `_` separando papeles (`keltnerUpperCrossUp_atrTrail`), sin activo ni temporalidad: una plantilla no pertenece a un mercado.

`registry.csv` — una fila por plantilla, clave `name` (`registry.py:12` , `:68`). Columnas: nombre, arquetipo, forma, entrada, salida, grupos, bloques, fecha, origen, estado, notas. El estado va `draft` → `validated` (el XML resuelve) → `buildConfirmed` (un build real sacó estrategias que llevan el bloque).

`runs.csv` — clave `(template, symbol, timeframe)` (`registry.py:14` , `:65`). Si la fila existe, **se actualiza en su sitio** (`append` , `:18-52`): no se añade otra. Hoy:

```
indicesIBS     USA500  H1   Test_USA500_H1_indicesIBS             2026-10-06   3000
doubleROCLong  USDJPY  M30  Test_USDJPY_M30_doubleROCLong_10k_B5  2026-10-08  10000
squeezeBBLong  XAUUSD  H1   Test_XAUUSD_H1_squeezeBBLong          2026-10-06   3000
```

Las columnas `strategies_kept` , `verdict` y `report` están vacías en las tres.

3.7 El registro de lo ya probado, y cómo evita repetir

Hay **cinco** sitios que saben algo de «esto ya se probó». Sólo uno impide repetir.

registro	clave	quién lo escribe	¿impide repetir?
`templates/runs.csv`	plantilla × activo × TF	la ventana tras un build (`ui/daemon/launch/run.py:113-122`), `/template-run` a mano, y `memory.report --close` para el veredicto	**no**: es para leerlo tú (y la zona Cobertura de la ventana)
`campaign/ledger.jsonl`	plantilla × activo × TF × dirección	la campaña, una línea por cambio de estado	**sí, salvo las de prueba**: `plan.py:67` y `:83` quitan de los candidatos toda celda registrada que no esté marcada `test`
`projects/registry.csv`	nombre del proyecto	el constructor, siempre	a medias: `plan.py:35-39` quita las celdas que ya corrió un proyecto `Trade_` o `Research_`
`ledger/<activo>_<TF>_<plantilla>.jsonl`	el estudio	cada paso que mira datos	no: **cuenta** (N para el Sharpe deflactado)
la memoria de investigación (`studies/research/memory/`)	proyecto	se regenera leyendo los otros	no: informa al director de investigación

Cómo evita repetir la campaña. `pipeline/campaign/plan.py`, función `candidates` (`:51-85`): cruza las celdas admitidas del tablero de investigación con las plantillas largas y sin avisos de su familia (`:24-32`), y descarta la celda si su clave está en el registro de la campaña **o** si un proyecto `Trade_`/`Research_` ya la corrió. Lo que queda se ordena por puntos del tablero y, al encolar, se congela como lote antes de que existan resultados.

El estado real hoy. El registro de la campaña tiene 54 líneas que se pliegan en **3 celdas**, las tres marcadas `test: true`: `indicesIBS|USA500|H1` (done), `doubleROCLong|USDJPY|M30` (done), `squeezeBBLong|XAUUSD|H1` (interrupted). No existe ningún lote congelado (`campaign/batches/` no está). Y `plan.py:67` excluye expresamente las celdas de prueba de lo «ya visto».

El mismo caso en los cinco registros — `doubleROCLong` × USDJPY × M30:

registro	lo que dice
`runs.csv`	1 fila: proyecto `…_10k_B5`, 10.000 construidas, sin veredicto
registro de la campaña	1 celda, `done`, de prueba → no bloquea
`projects/registry.csv`	**11 altas de proyecto** (7 nombres distintos: `…_doubleROCLong` cuatro veces y `…_10k` dos; del 2026-10-05 al 2026-10-08), todos `Test_` → no bloquean
libro de búsqueda	**8 builds**: 593.788 + 162.804 + 148.217 + 832.137 + 749.129 + 777.781 + 784.406 + 832.299 = **4.880.561 generadas**, 324.271 aceptadas; más 75 filas de pasos posteriores: 22 sobre `build`, 29 sobre `oos1` y **24 sobre `oos2`** (detalle debajo)
almacén	41 estrategias guardadas

Es decir: lo que tú leerías en `runs.csv` («una corrida de 10.000») y lo que el libro de búsqueda cobra («ocho builds, 4,9 millones») son la misma celda.

Qué tramos de datos ha consumido la celda, según el libro. (Hallazgo del revisor Taleb; las filas las he releído yo. Es lo que el libro *anota*: ni él ni yo hemos abierto cada estudio para ver qué calculó sobre cada tramo, y no he recalculado nada sobre `oos2`.)

tramo	qué anota el libro
`build`	8 builds, 4.880.561 generadas. Sus filas tienen vacíos `window_from`, `window_to` y `config_hash`: el libro no permite comprobar que los ocho usaron la misma ventana ni la misma configuración
`oos1`	el paso 8 lo cribó **4 veces** (2026-10-05 23:52, 10-06 15:34, 10-06 20:09, 10-07 17:28) con **4 `config_hash` distintos** (`2f460bf9`, `2b8101bb`, `1645c00c`, `8346370d`; según el revisor ese hash es el sha256 de `criteria.yaml`, `pipeline/autopilot/judge.py:150`). Entraron 3.000 + 3.000 + 3.000 + 10.000 = **19.000** estrategias y salieron 5, 3, 3 y 56. Esas filas tienen `n_scored` vacío: no suman a N
`oos2` (el reservado)	**24 filas**, todas del 2026-10-06 (16-17 h y 21 h): pasos 17 (6), 18 (6), 18.5 (4), 24 (4), 19 (2) y 20 (2). Las dos del paso 20 son de `studies.closing.blindJoint.report`, ventana 2023-01-01 → 2026-10-01, 3 estrategias cada una, 0 salen

Es decir: para la campaña la celda está sin probar; para los datos, su `oos1` se ha mirado cuatro veces con el criterio cambiando entre mirada y mirada, y su `oos2` ya está abierto dos veces.

3.8 Cuánto ensancha o estrecha el espacio cada hueco

Aplico la cuenta de 2.5 a números reales. **Calculado por mí** (`space.py`), con los supuestos de 2.5. C es el número de condiciones encendidas en la tarea Build.

Lógicas de entrada distintas, por forma y por paleta:

forma	C = 103 (paleta de `doubleROCLong`)	C = 400 (el donante sin paleta)	C = 575 (todas)
2 fijas, sin hueco	1	1	1
1 fija + hueco 0..1	**104**	401	576
atado a grupo de g + libre 1..1	g × 103: de 206 (g = 2) a 1.339 (g = 13)	g × 400	g × 575
Simple Strategy o un hueco 1..2 (sólo entrada)	5.356	80.200	165.600
dos huecos 1..2 (rechazada), como conjuntos de 2 a 4 bloques	4.603.379	1.061 millones	4.539 millones

Lecturas:

- **La paleta manda más que la forma mientras haya un solo hueco.** Pasar de 103 a 575 condiciones multiplica las lógicas por 5,5 con un hueco 0..1 y por 31 con un hueco 1..2.
- **Un segundo bloque en el mismo hueco eleva al cuadrado.** De 0..1 a 1..2 con la misma paleta: de 104 a 5.356, cincuenta veces más. Es la razón numérica de la regla 14.
- **Un hueco atado multiplica por el tamaño de su grupo, y la paleta no lo puede estrechar.** Los grupos de las 18 plantillas «de grupo»: `MeanReversionFilters` 2 bloques; `StructuralBreakFilters` y `VolatilityRegimeFilters` 4; `TrendRegimeFilters` y `DirectionalMomentumFilters` 6; `BookTriggers` 13 (12 plantillas entre los seis). Las otras seis: `entropyEMALong` / `Short` y sus `_withExit`, atadas a `EntropyFilter` (2 bloques); y `acceleratorEMALong` y su `_withExit`, atadas a `AcceleratorFilter` (4 bloques; identificador `2e18e481-…`, no instalado en SQX_w1).
- **Un hueco de salida multiplica otra vez.** Una `_withExit` (hueco libre de salida 0..2) con paleta de salida de 103: `1 + 103 + 5.253` = 5.357 salidas posibles por cada entrada.

Y los números, que son el grueso. Sólo con lo que la plantilla `doubleROCLong` fija: dos periodos de ROC × `ExitAfterBars` de 4 a 48 (45 valores) × 104 lógicas, sin contar los parámetros propios del bloque del hueco. El resultado depende por completo de la rejilla de periodos, y **no sé cuál es** (5.9):

rejilla de periodos supuesta	parejas sin orden	× 45 × 104	los 4.880.561 generados son
2 a 200 de uno en uno (lo escrito en la tarea), parejas iguales incluidas	19.900	93.132.000	el 5,2 %
pares de 10 a 120 (lo observado en las 41 guardadas), sin parejas iguales	1.540	7.207.200	el 68 %

La segunda fila es cuenta de los revisores, que he rehecho (con parejas iguales serían 1.596 y el 65 %). Ninguna de las dos es una cota: la primera supone una rejilla que las guardadas no apoyan, y la segunda toma como rejilla lo que muestran 41 supervivientes, que puede ser sesgo de la criba y no el rango del sorteo. Si la rejilla real se parece a la segunda, ocho builds han barrido buena parte de ella y parte de las 4,9 millones pueden ser la misma estrategia repetida; no sé cuántas (el registro de proyectos dice que los lotes `_L2` y `_B2` eran una prueba de duplicados entre lotes; no he encontrado su resultado).

Qué suma el libro, y qué no. `ledger/trials.py:13-58`: N es la suma de `n_scored` de todas las búsquedas del estudio, y el paso 6 anota como `n_scored` las **estrategias generadas** que SQX dice en su log. Para esta celda `trials.accumulated()` devuelve hoy `n = 4.880.567` (las generadas más las 6 del

paso 20). Conviene no mezclar las tres N:

N	valor en la celda	qué es	quién lo calcula
lógicas posibles	104	tamaño del espacio que dejan plantilla y paleta; no son pruebas hechas	nadie: no es un hecho ni una fila del libro (la cuenta es mía)
generadas	4.880.561	lo que compitió en el tramo de build	el libro, paso 6
N efectiva	sin medir sobre las generadas	cuántas se comportan distinto	`population_n_eff` (`trials.py:78`) agrupa el panel de la población que se le pase, que a estas alturas son supervivientes: no puede decir cuántas cosas distintas había entre las generadas

El código recomienda dar el N bruto y el agrupado: el bruto exagera la penalización, el agrupado la rebaja. Cuál es el correcto no lo decide este documento.

---

4. El criterio cargado y por qué

4.1 En `criteria.yaml`, `facts.py`, `thresholds.yaml`

Nada. El paso 3 no produce hechos y no tiene reglas. No es un olvido: es autoría.

4.2 Las negativas del constructor

regla	qué mira	negativa	`fichero:línea`	en la librería hoy
13, una dirección	reglas `Long entry` / `Short entry` con acción `Enter…`	si no es exactamente una	`builder.py:34-42`, `:213-218`	5 rechazadas
14, dos huecos libres	`free > 1` en un lado	siempre	`buildrules.py:158-161`	0
14, tope de 2	`fijas + atadas + libres > 2` en un lado	siempre	`buildrules.py:162-165`	3 (dos de ellas ya rechazadas por la 13)
nombre del proyecto	prefijo `Test_` / `Trade_` / `Research_`	siempre	`builder.py:207-208`	—
el fichero existe	—	siempre	`builder.py:209-210`	—

Total: **6 de 61** inconstruibles (`gbpRandomEntry`, `gbpRandomEntryExit`, `gbpRandomExit`, `gbpjpyEndMonth`, `spRSI2`, `xauWeekend`).

4.3 Las comprobaciones al emitir

`build.py:172-174`: el XML se lee, la clave del bloque está, los huecos son dos. Nada más.

4.4 Lo que filtra la campaña sin ser negativa

`plan.py:30`: una plantilla con algún aviso en el campo `flags` de su `manifest.json` no entra en los candidatos. Hoy llevan aviso 10: las seis de arriba; `acceleratorEMALong` y su `_withExit` («el grupo no está instalado en SQX_w1»); `squeezeBBShort` y su `_withExit` («posible defecto sin comprobar: la señal se escribe en la variable del largo»). Además la campaña sólo toma largas (`config.yaml`).

4.5 Por qué estas reglas

- **Una dirección** (2026-10-01): una idea corta es otra plantilla; mezcladas, no se sabe de qué lado viene el resultado.
- **Dos condiciones por lado** (2026-10-02, «fundamental»): cada condición de más multiplica el espacio (3.8) y, con él, lo fácil que es encontrar algo por azar.
- **Fijar lo nombrado, sortear lo no nombrado**: parametrizar lo que nombraste ensancha la búsqueda con variantes que ya habías descartado; sortear periodos es precisamente el punto.

---

5. Lo que no cuadra o no está medido

5.1 La aritmética de la regla 14 no está en el commit

En `b2b4c8c`, `builder.py:219` llama a `buildrules.over_the_cap`, pero el `buildrules.py` confirmado no tiene esa función ni `what_to_build`: escribe `minConditions = 1`, `maxConditions = 2` para todo. `holes.conditions()` tampoco está confirmada, y su prueba (`tests/test_condition_count.py`) no está añadida a git. Un clon limpio del repositorio falla al crear cualquier proyecto que pase `--template` (los de plantilla y también el paso 26, que entra por ahí con una estrategia terminada); los `--simple` no pasan por esa línea. Todo lo que este documento describe es el árbol de trabajo de hoy.

5.2 La librería actual no salió del camino que describe el paso

Las 61 plantillas son tus ficheros, importados «byte a byte» el 2026-10-05 por `scratch/tplimport/run.py`, un script desechable que no está en git; ese script es también quien escribió los `manifest.json` y sus avisos. Las ocho plantillas que sí emitió `sqx.templates.build` (`keltnerUpperCrossUp`, `donchianUpperCrossUp`, `timeRangeBreakout`, `freeShellLong`…) están en `_retiradas-2026-10-05/`. Consecuencias:

- la skill y `sqx/templates/README.md` dicen «copia la estructura de `keltnerUpperCrossUp`», que ya no está en la librería;
- la ficha `condition-count-simple-vs-template.md` dice en su evidencia que «toda plantilla escrita es `{bound 1, free 1}` en la entrada y nada en la salida»: describe la librería anterior. Hoy 39 son `{fixed 1, free 1}` (37 de ellas construibles) y 29 tienen hueco de salida;
- si los `manifest.json` hay que regenerarlos (y la campaña lee de ellos el recuento, `plan.py:77`, y los avisos, `:30`), no hay comando en el repositorio que lo haga.

5.3 Ninguna plantilla llega a `buildConfirmed`

61 de 61 en `draft`. `doubleROCLong`, `indicesIBS` y `squeezeBBLong` se han construido de verdad. El estado lo cambian dos sitios, los dos a mano: la skill `/template-run` (`.claude/skills/template-run/SKILL.md:140`), que antes pasa `sqx.inspect.template_check` (¿las estrategias llevan el bloque fijo?),

y la ventana (`ui.daemon.writes.set_status`, `ui/daemon/writes.py:10-30`, expuesta en `ui/daemon/templates.py:101`), que lo mueve sin comprobar nada. Ni `bin/workflow.sh`, ni el autopiloto, ni la campaña llaman a ninguna de las tres cosas. Mi comprobación sobre las 41 guardadas (3.3) dice que sí lo llevan, pero es mía y sobre supervivientes.

Relacionado: `sqx/CLAUDE.md:82-83` dice de `template_check` que comprueba «si las estrategias que construyó un proyecto llevan los bloques que su plantilla fija — en esta instalación, no los llevan». Esa frase es anterior a los arreglos; en lo que he mirado, los llevan.

5.4 «Registro de lo ya probado»: cinco libros que no coinciden

- **`runs.csv` pierde la historia.** Clave plantilla × activo × TF y actualización en el sitio: once altas de proyecto (siete nombres) sobre `doubleROCLong` × USDJPY × M30 son una fila, la del último que escribió. `AlgoData/templates/README.md` lo describe como «una fila por (plantilla × símbolo × timeframe × corrida)»; el código no tiene la corrida en la clave.
- **Nadie rellena el veredicto.** `memory.report --close` sólo se llama desde la cola de investigación de la ventana (`ui/daemon/research/steps.py:115`); el camino de `bin/workflow.sh` y el de la campaña no cierran la fila. Las tres filas tienen el veredicto vacío.
- **Lo que impide repetir no ve las pruebas.** El registro de la campaña excluye sus celdas `test` y `_launched()` sólo mira `Trade_` y `Research_`. Las tres celdas con builds reales son todas `Test_`: para la campaña, **ninguna celda está probada**.
- **El manual lo avisa a su manera** (`manual-fuentes/22-plantillas.md:185`): «`registry.csv` no se lee solo… si no anotas una corrida, la respuesta a "¿ya lo probé?" será que no».

Qué falta para poder escribir una regla: decidir **cuál** es el registro que manda y qué cuenta como «probado» (¿un build?, ¿un build que llegó al paso 8?, ¿también los `Test_`?).

5.5 Un intento que falla no queda en ningún «ya probado»

`Test_EURUSD_M30_filterMALong_10k` (2026-10-07): el build abortó en un segundo porque la plantilla lee un segundo gráfico y nuestros proyectos definen uno (ficha `template-second-chart-aborts-build.md`). Está en `projects/registry.csv` y en la carpeta del autopiloto; no en `runs.csv`, ni en el libro de búsqueda, ni en la campaña. La ficha dice que cinco plantillas tienen ese problema (`filterMALong`, `filterMAShort`, sus `_withExit` y `spRSI2`) y que **el constructor no las rechaza de antemano**. Cuatro de ellas no llevan aviso en su `manifest.json`: las dos largas (`filterMALong` y su `_withExit`) entrarían en los candidatos de la campaña.

5.6 La regla 15 no se comprueba sobre lo que la plantilla fija

El filtro de reloj y calendario (`sqx/projects/buildcensus.py`, `CLOCK`) actúa sobre los bloques de la **paleta**, es decir, sobre lo que puede caer en un hueco libre. No he encontrado en `builder.py`, `buildrules.py` ni `sqx/templates/` ninguna comprobación sobre un bloque de calendario **clavado** en la plantilla. Las cinco que lo tienen (`gbpRandomEntry`, `gbpRandomEntryExit`, `gbpRandomExit`, `xauWeekend` y `gbpjpyEndMonth`) se rechazan hoy por otras reglas; una plantilla larga, de una dirección, con «es lunes» fijo y un hueco se aceptaría.

5.7 El código es más laxo que el texto de la regla 14 en dos puntos

- **Un solo hueco libre y nada fijo se acepta** (fila 6 de la tabla), con rango 1..2. La regla dice que eso es una Simple Strategy y que no se escribe plantilla. Ninguna de la librería actual tiene esa forma en la entrada.
- **«Gemelas» es más ancho que «dos periodos».** La huella que decide si dos fijas son la misma idea es la lista de claves de bloque, sin mirar parámetros (`holes.py:80`). Dos `ROCAboveLevel` con niveles distintos (ROC > 0 y ROC > 5), o tres o cuatro copias, también contarían como una.

5.8 Un hueco atado a un grupo de varios bloques se cuenta como idea fija

`buildrules.py:128-129` y `:139-140` tratan `bound` igual que `fixed` para el tope, y fuerzan 1..1. Eso es exacto para un grupo de un bloque (lo que emite `build.py`). En las 18 plantillas «de grupo» el grupo tiene de 2 a 13 bloques: es un segundo sorteo, estrecho y que la paleta no alcanza. El recuento de condiciones sigue siendo correcto (salen 2); lo que cambia es el espacio (3.8) y el nombre que se le da.

Y sobre el grupo de un bloque, la ficha `one-item-group-can-drift.md` (□ 2026-09-27): fija la **primera** elección, pero la mutación genética puede mover el hueco después; se midió 197 de 200.

5.9 Lo que no está medido

- **Con qué frecuencia el hueco 0..1 queda vacío** en la población construida (no en las supervivientes). Decide si «1 fija + hueco» produce también la idea desnuda.
- **Si el rango 1..1 con un hueco atado se comporta como se espera.** Nunca se ha construido una plantilla así con ese rango (la última con hueco atado, `donchianUpperCrossUp`, es del 2026-10-02 04:15, antes del arreglo).
- **Si el rango de salida 0..2 y `_ExitRule_` al 50 % hacen algo cuando la plantilla no tiene hueco de salida.** 41 de 41 guardadas sin condición de salida; no es la población entera.
- **De dónde sale el rango efectivo de periodos** (10 a 120, pares, observado) frente al `minPeriod=2 maxPeriod=200` escrito en la tarea.
- **Si SQX puede poner dos veces el mismo bloque en un hueco 1..2.** Cambia la cuenta de 2.5 en C opciones.
- **Cuántas lógicas distintas tiene de verdad una población construida.** En las 41 guardadas son 3. Sobre las 10.000 de un build no lo he contado: sus `.sqx` no se archivan.
- **El N «de lógicas».** No existe como hecho ni como fila del libro de búsqueda.

5.10 Formas «confirmadas en build»: dos listas que no son la misma

La tabla de la skill (texto del proveedor) marca como confirmadas `stop`, `session_market`, `two_entry_market`, `stop_long`, `market_long` y `mtf_filter_long` — en la instalación del proveedor, en junio de 2026 — y como pendientes `market`, `market_single`, `role_market`, `mtf_filter`, `market_short`, entre otras. De ésas, **en este proyecto** varias están prohibidas por regla: las de entrada stop (regla 15 mientras las órdenes sean a mercado), `session_market` (reloj), `two_entry_market` / `multi_leg` / `role_market` (más de dos condiciones o varias patas), `mtf_filter*` (segundo gráfico, 5.5).

Lo construido **aquí**, según `projects/registry.csv`:

forma	plantillas con proyecto	estado
1 fija literal + 1 hueco libre, larga, a mercado	`indicesIBS` (5 proyectos), `squeezeBBLong` (4), `doubleROCLong` (11, con fija gemela)	construidas con el rango nuevo (0..1)
1 atada a grupo de un bloque + 1 libre, larga	`donchianUpperCrossUp` (16), `emaCloseCrossUp` (2), `timeRangeBreakout` (5; ésta con fija literal)	construidas antes del arreglo del 2026-10-02 (rango 1..2); retiradas de la librería
dos huecos libres	`freeShellLong` (4), `freeShellShort` (3)	construidas; hoy rechazadas
Simple Strategy larga	`simple:long` (6)	construida
cualquier forma corta de la librería actual	—	**ninguna construida**
cualquier `_withExit`	—	**ninguna construida**
1 atada a grupo de varios + 1 libre	—	**ninguna construida**

5.11 Dos frases de la documentación que ya no valen

- `sqx/templates/README.md:23` y la skill: el hueco libre por defecto «muestrea todo el vocabulario de 500 condiciones». La tarea Build tiene 575 condiciones, y la regla 15 prohíbe «la paleta entera»: el hueco libre muestrea lo que la paleta del paso 6 encienda.
- `CLAUDE.md`, regla 14, no menciona el caso «atado + libre → 1..1», que es el de 18 plantillas. Sí está en la ficha de conocimiento y en `sqx/CLAUDE.md` no.

5.12 El dossier contaba builds; el libro dice además qué tramos se han gastado

(Falta señalada por el revisor Taleb; tabla en 3.7.) En la celda de los ejemplos, `oos1` se ha cribado cuatro veces con cuatro versiones de `criteria.yaml` (19.000 estrategias entre las cuatro) y el libro anota 24 filas sobre `oos2`, dos de ellas del juicio ciego del paso 20. Ningún registro de «ya probado» del paso 3 recoge eso: `runs.csv` no tiene columna de tramo, y la campaña ve la celda como sin probar. Para un resultado nuevo de esta plantilla en este activo, `oos1` ya no es fuera de muestra respecto de los criterios. Salvedad del revisor, que mantengo: es lo que el libro anota; no se ha abierto cada estudio.

5.13 El N del libro va con una dispersión de seis datos, y el autopiloto no lo lee

(Calculado por el revisor; la primera cifra la he reproducido llamando a la función.) `trials.accumulated()` sobre este estudio: `n = 4.880.567`, `sigma = 0,0265` por periodo, `searches = 2`. Las ocho filas de build llevan `score_unit = "generated"` y ningún Sharpe, así que la dispersión sale **sólo** de las dos filas del paso 20: seis Sharpes de supervivientes. El listón del Sharpe deflactado es esa dispersión por una función de N (`core/surface/plateau.py:55-71`): N viene de 4,9 millones y σ de seis. Y a `accumulated()` la llaman `ledger/report.py`, `portfolio/common/construct/oos2.py`, `core/archive/manifest.py` y la ventana (`ui/daemon/ops/ledgerview.py`); ningún fichero de `pipeline/autopilot/` (lo he comprobado con `grep`). Es decir: el recuento existe, pero los pasos 6 a 25 no deciden con él.

5.14 El N es por plantilla, no por serie de precios

(Falta señalada por el revisor.) El estudio del libro es activo × TF × plantilla. Otra plantilla sobre el mismo USDJPY M30 empieza de cero aunque mire los mismos precios. Los otros seis libros de USDJPY M30 (`donchianUpperCrossUp`, su `Test_…`, `freeShellLong`, `freeShellShort`, `simple_long`, `simple_short`)

no tienen ninguna fila de build (comprobado: 0 filas de paso 6 en los seis): lo que se generó allí no está contado en ningún sitio. Según el revisor, además, dos de las once altas de proyecto de esta celda (la del 2026-10-06 08:54 y `_10k_L2`) no tienen fila de build, y `trials.overall()` sobre los 93 estudios da 11.512.094; esas dos cifras no las he rehecho. A favor del libro: a diferencia de la campaña, **sí cuenta las pruebas** `Test_` (las ocho filas de build son de proyectos `Test_`), así que una corrida `Trade_` de la campaña sobre esta celda heredaría ese N.

---

6. Cierre

6.1 Las reglas, en una tabla

qué	regla	dónde se hace cumplir
dirección	exactamente una	al crear el proyecto
huecos libres por lado	como mucho uno	al crear el proyecto
fijas + atadas + libres por lado	como mucho 2 (gemelas cuentan una)	al crear el proyecto
rango del hueco	`0..(2 − fijas)`; 1..1 si hay un atado	al escribir la tarea Build
reloj o calendario fijo en la plantilla	prohibido por la regla 15	**en ningún sitio**
segundo gráfico	rompe el build	**en ningún sitio**
«ya probada aquí»	—	sólo en la campaña, y no para pruebas

6.2 El resultado sobre la librería real

61 plantillas: 55 construibles (37 «fija + hueco», 18 «grupo + hueco»; 27 de ellas con hueco de salida), 6 rechazadas. 4 con proyecto alguna vez; 3 con build terminado; 0 marcadas como confirmadas. 10 con aviso (fuera de la campaña) y al menos 4 más que abortarían sin aviso.

6.3 Las decisiones que te quedan

1. **Qué registro manda para «ya probado», y qué cuenta como probado.** Gana: una sola respuesta a «¿ya lo hice?», y que la campaña no repita celdas que ya consumieron millones de estrategias. Cuesta: decidir si los `Test_` cuentan (hoy, para la campaña, `doubleROCLong` × USDJPY × M30 está sin probar con 4,9 millones generadas en el libro, `oos1` cribado cuatro veces y 24 filas sobre `oos2`) y, si `runs.csv` va a ser la historia, cambiar su clave para que no pise.

2. **Qué N se cobra a una celda: estrategias generadas, lógicas distintas, o las dos.** Gana: una corrección que distinga «104 ideas con muchos números» de «165.600 ideas». Cuesta: hoy el N de lógicas no se calcula; hace falta que el paso 5 o el 6 lo anote (plantilla + paleta lo determinan antes del build). Con N = generadas el valor esperado del mejor por azar es 5,2 desviaciones (cota 5,5); con N = 104, 2,5 (cota 3,0); ilustrativo, para ruido independiente y normal. **No son un intervalo**: no está demostrado que el valor correcto caiga entre los dos. El parecido entre vecinas lo empuja hacia abajo (calculado por el revisor: 38 guardadas de una misma lógica ≈ 2 independientes, medido sólo en supervivientes), y la asimetría de las operaciones y las búsquedas de otras plantillas sobre el mismo USDJPY M30 lo empujan hacia arriba. Nadie ha medido cuál domina.

3. **Las «gemelas»: ¿hasta dónde llega «la misma condición dos veces»?** Gana: que la excepción diga lo que quieres decir. Cuesta: hoy admite cualquier número de copias y parámetros distintos del periodo. Si la quieres estricta (exactamente dos, sólo cambia el periodo), hay que escribirlo; ninguna plantilla actual quedaría fuera.

4. **Las 18 plantillas «de grupo»: ¿idea fija o segundo hueco?** Gana: llamar a las cosas por su nombre en el recuento y en el espacio. Cuesta: si son un segundo hueco, tocan la frase «nunca dos huecos en un lado»; si son idea fija, conviene saber que la paleta no las estrecha y que multiplican el espacio por 2 a 13. Antes de usarlas hay que construir una: la forma 1..1 no se ha probado nunca.

5. **Comprobar al crear el proyecto lo que hoy no se comprueba: calendario fijo y segundo gráfico.** Gana: que una plantilla inconstruible o contra la regla 15 falle en un segundo y con su motivo, en vez de abortar el build o colarse. Cuesta: cuatro plantillas más pasarían a «rechazada» (`filterMALong`, `filterMAShort` y sus `_withExit`), salvo que se decida darles su segundo gráfico.

6. **Quién marca una plantilla como confirmada.** Gana: que `status` vuelva a significar algo (hoy 61 de 61 son `draft`). Cuesta: enganchar `template_check` y el cambio de estado al workflow de un solo comando, o aceptar que la columna no se usa y quitarla.

7. **Confirmar el árbol de trabajo.** Gana: que la regla 14 exista en el repositorio y no sólo en este disco. Cuesta: nada de diseño; es tu «sí» al commit de `holes.py`, `buildrules.py`, la prueba y `pipeline/campaign/`.

6.4 El límite que no cambia

La plantilla decide **qué** se busca; no dice nada de si lo encontrado vale. Estrechar el hueco reduce las lógicas que compiten, pero los millones de juegos de números siguen ahí: una plantilla estrecha construida ocho veces sobre la misma ventana no es una búsqueda pequeña. El recuento de condiciones limita la complejidad de cada estrategia; el recuento de intentos —por plantilla, por activo, por temporalidad, incluidos los fallidos y los de prueba— es lo que limita la del conjunto, y hoy es el que está peor llevado.